

EXPERIMENT 1

DESIGNING A MINIMUM DISTANCE TO CLASS MEAN CLASSIFIER

Submission Date: 11 June, 2016

Student ID: 12.02.04.026

Section: A1

Year and Semester: 4.2

Ahsanullah University of Science and Technology
Department of Computer Science and Engineering

Contents

Introduction	2
Objective of the Experiment	3
Performing the experiment	3
Plotting two class set of given prototypes	4
Classifying given points with given linear discriminant function	5
Drawing the decision boundary between two classes	6
Result Analysis	8
Conclusion	9

INTRODUCTION

The minimum distance to class mean classifier defines classes in terms of the distance from the mean vector for the class. In this type of classifier the mean point for the sample points of known classes are at first calculated, and then the sample points of unknown class are assigned to the class for which the distance to class mean is the minimum.

Suppose the mean of each class is given by,

$$m_j = \frac{1}{N_j} \sum x_j \quad [j = 1, 2, \dots, W]$$

where N_j is the number of sample points from class ω_j and W is the number of classes. Now one way to determine the class label of an unknown sample point, x is to assign it to the class of its closest class mean, as noted earlier. Using Euclidean distance to determine closeness we have,

$$D_j(x) = \|x - m_j\| \quad [j = 1, 2, \dots, W]$$

We then assign x to ω_i if $D_i(x)$ is the smallest distance. Since selecting minimum to the function -

$$d_j(x) = x^T m_j - \frac{1}{2} m_j^T m_j \quad [j = 1, 2, \dots, W]$$

Comparing the above equation with the equation of linear discriminant function -,

$$g_i(x) = w_i^T x + w_0$$

where

$$w_i^T x = x^T m_j$$

and

$$w_0 = \frac{1}{2} m_j^T m_j$$

Hence we may conclude from the above discussion that the Minimum distance to mean classifier is a linear classifier.

OBJECTIVE OF THE EXPERIMENT

The objective of the experiment is to design a minimum distance to class mean classifier given the following two class set of prototypes

$$\omega_1 = \{(2, 2), (3, 1), (3, 3), (-1, -3), (4, 2), (-2, -2)\}$$

$$\omega_2 = \{(0, 0), (-2, -2), (-1, -1), (-4, 2), (-4, 3), (2, 6)\}$$

1. Plot all sample points from both classes, but samples from the same class should have the same color and marker.

2. Using a minimum distance classifier with respect to "class mean", classify the following points by plotting them with the designated class-color but different marker.

$$X_1 = (-1, -1)$$

$$X_2 = (3, 2)$$

$$X_3 = (-2, 1)$$

$$X_4 = (8, 2)$$

Linear Discriminant Function:

$$g_i(X) = X^T \bar{Y}_i - \frac{1}{2} \bar{Y}_i^T \bar{Y}_i$$

3. Draw the decision boundary between the two-classes.

PERFORMING THE EXPERIMENT

Plotting two class set of given prototypes

At first I plotted the given sample points of both classes ω_1 and ω_2 . The points of same class were plotted using the same color and marker. For ω_1 I used the color blue and the marker diamond whereas for ω_2 I used the color red and the marker star.

Matlab Code 1.1

```
1: w1 = [2 2; 3 1; 3 3; -1 -3; 4 2; -2 -2];
2: w2 = [0 0; -2 2; -1 -1; -4 2; -4 3; 2 6];
3: figure
4: plot(w1(:,1), w1(:,2), '*', 'LineWidth', 2, 'MarkerEdgeColor', 'b', 'MarkerFaceColor', 'b');
5: hold on;
6: plot(w2(:,1), w2(:,2), 'd', 'LineWidth', 1, 'MarkerEdgeColor', 'r', 'MarkerFaceColor', 'r');
7: legend('w1', 'w2', 'Location', 'northwest');
```

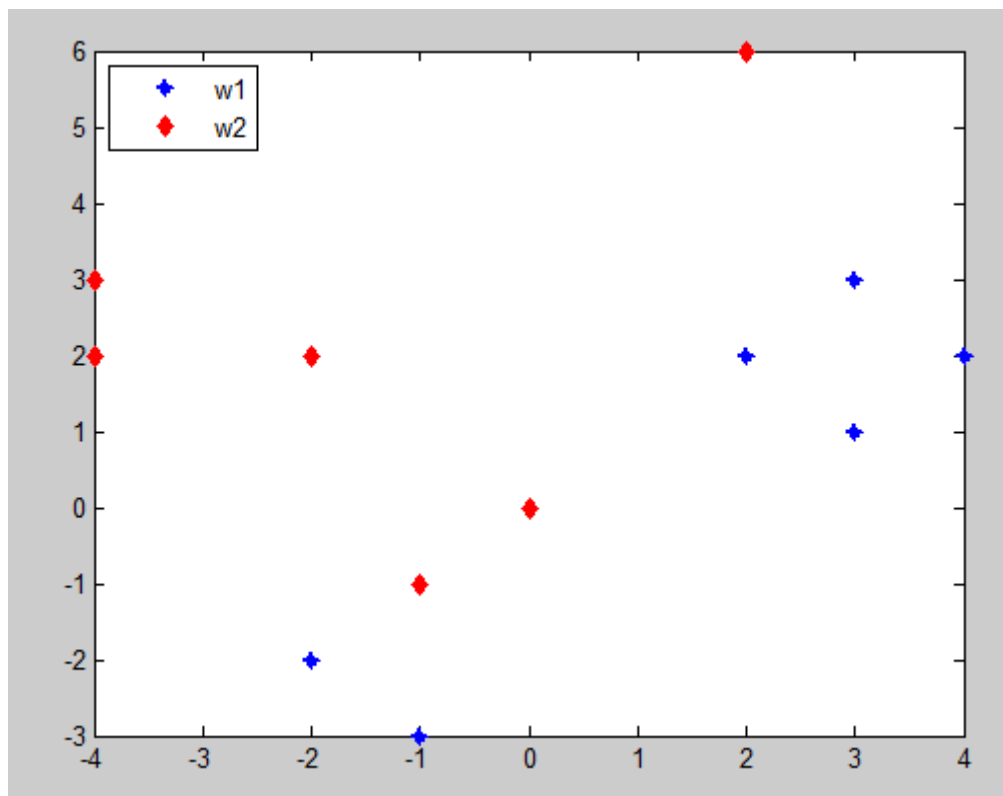


Figure 1: Output after running Matlab code 1.1

Classifying given points with given linear discriminant function using designated class color but different marker

At first the mean points of each of the classes are calculated. Then for each of the points (X_1, X_2, X_3, X_4), the distance from the mean is calculated using the given discriminant function -

$$g_i(X) = X^T \overline{Y}_i - \frac{1}{2} \overline{Y}_i^T \overline{Y}_i$$

Next the points were assigned to the class with minimum distance from the respective class mean. I used the same class color but different marker (squares) while plotting the points.

Matlab Code 1.2

```

1: y1 = [mean(w1(:,1)) mean(w1(:,2))];
2: y2 = [mean(w2(:,1)) mean(w2(:,2))];
3:
4: x = [-1 -1; 3 2; -2 1; 8 2];
5:
6: for n=1:4
7:     g1 = x(n,:)*y1' - .5*(y1*y1');
8:     g2 = x(n,:)*y2' - .5*(y2*y2');
9:     if g1>g2
10:         plot(x(n,1),
11:             x(n,2),'s','LineWidth',2,'MarkerEdgeColor','b','MarkerFaceColor','b');
12:     else
13:         plot(x(n,1), x(n,2),'s','LineWidth',2,'MarkerEdgeColor','r','MarkerFaceColor','r');
14:     end
15: end

```

output of the above code is shown in the figure next page.

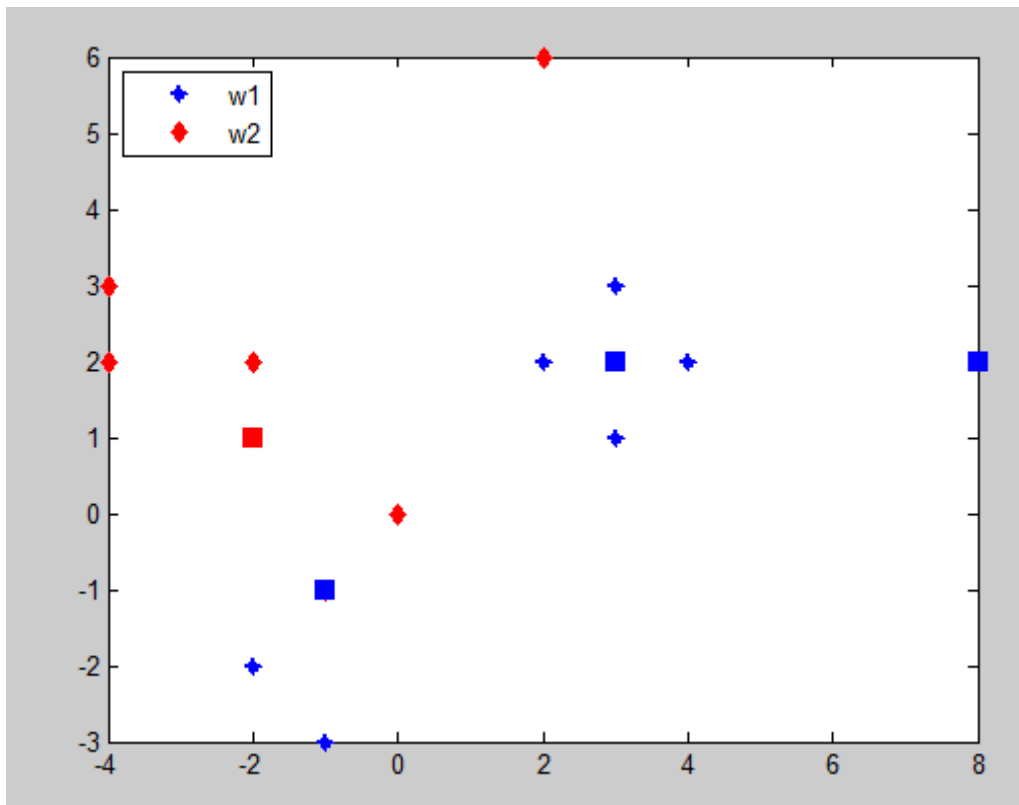


Figure 2: Output after running Matlab code 1.2

Drawing the decision boundary between two classes

Decision boundary between the two classes is to be drawn now. We know that all points on the decision boundary will be at equal distance from ω_1 mean and ω_2 mean.

$$\begin{aligned}
 g(x) &= g_1(x) - g_2(X) \\
 &= \overline{w_1}^T x - \frac{1}{2} \overline{w_1}^T \overline{w_1} - \overline{w_2}^T x + \frac{1}{2} \overline{w_2}^T \overline{w_2} \\
 &= (\overline{w_1}^T - \overline{w_2}^T) x - \frac{1}{2} (\overline{w_1}^T \overline{w_1} - \overline{w_2}^T \overline{w_2})
 \end{aligned}$$

Now since on decision boundary the points are at a equal distances, we have,

$$\begin{aligned}
& (\overline{w_1}^T - \overline{w_2}^T)x - \frac{1}{2}(\overline{w_1}^T \overline{w_1} - \overline{w_2}^T \overline{w_2}) = 0 \\
& \Rightarrow \begin{bmatrix} COEF_1 & COEF_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + CONSTANT = 0 \quad [CONSTANT = -\frac{1}{2}(\overline{w_1}^T \overline{w_1} - \overline{w_2}^T \overline{w_2})] \\
& \Rightarrow COEF_1 \times x_1 + COEF_2 \times x_2 + CONSTANT = 0 \\
& \Rightarrow x_2 = \frac{COEF_1 \times x_1 + CONSTANT}{-COEF_2}
\end{aligned}$$

Matlab Code 1.3

```

1: d1 = [-4:2:8];
2: coef = (y1-y2);
3: constant = 0.5 * det((y1*y1') - (y2*y2'));
4: for i=1:length(d1)
5:     d2(i,:) = (coef(1,1)*d1(:,i)+constant)/(-coef(1,2));
6: end
7: d = [d1' d2];
8: hold on;
9: plot(d(:,1),d(:,2), 'r-', 'LineWidth',2,'MarkerEdgeColor','r','MarkerFaceColor','r');
10: hold off;

```

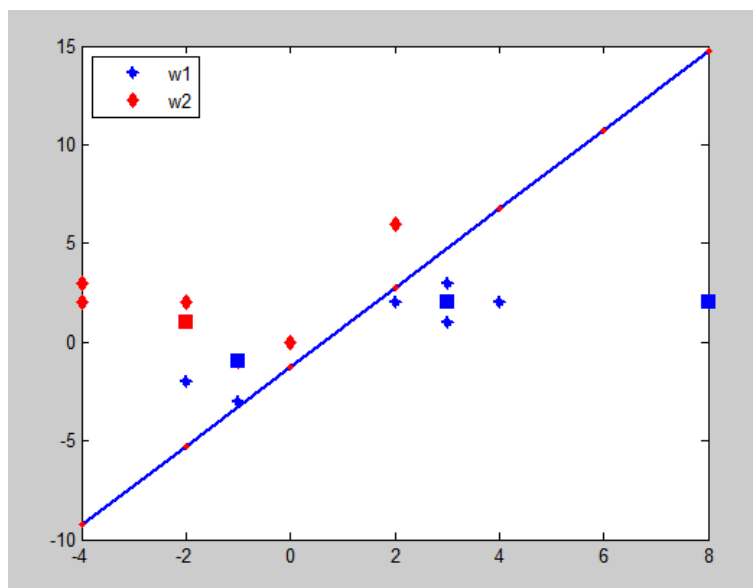


Figure 3: Output after running Matlab code 1.3

RESULT ANALYSIS

Here we see that out of 4 sample data, 1 was misclassified and so the accuracy obtained is 75%

Since minimum distance to mean classifier is a linear classifier, it does not do a very good job if the sample points of each class are uniformly distributed/scattered in the feature space. However the classifier will give good result if the sample points of one class is far from the other.

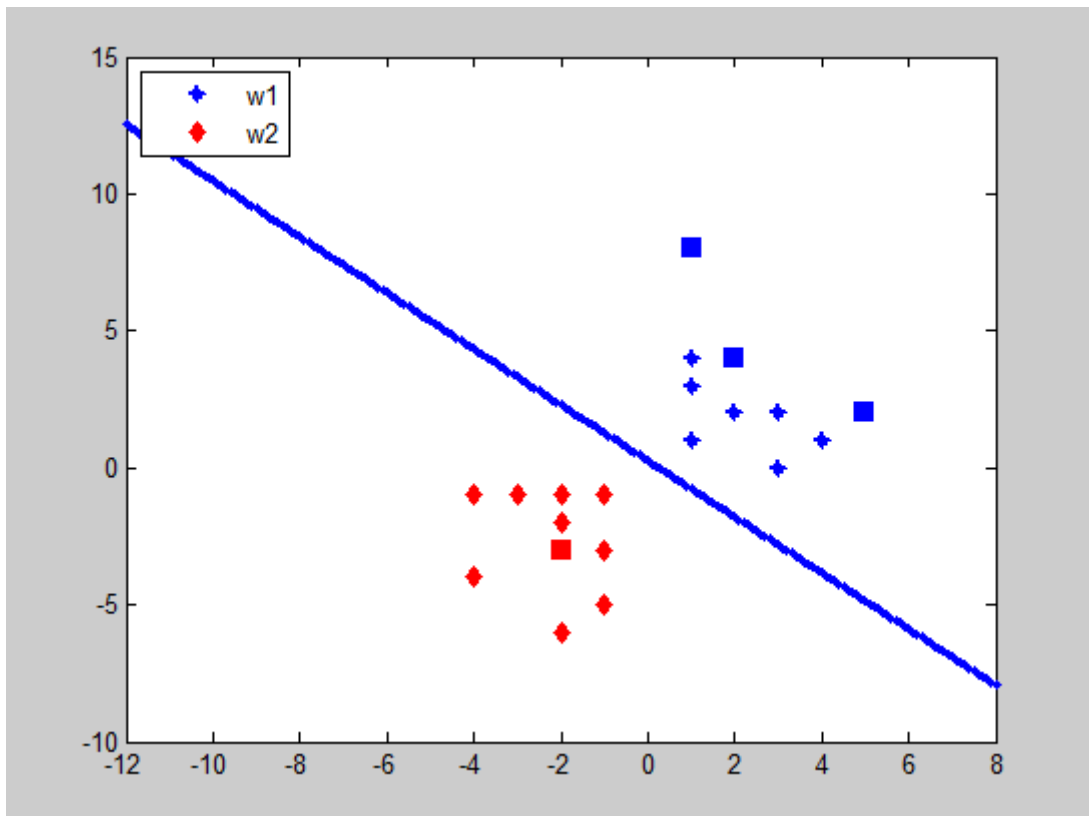


Figure 4: One of the best cases of this classifier with accuracy 100%

Figure 4 has 100% accuracy because the points of each class has a fair distance from each other and the sample points too are close to the mean of the class to which it belongs. the dataset used for this is

$$\omega_1 = \{(2, 2), (3, 0); (3, 2); (1, 3); (4, 1); (2, 4); (1, 1); (2, 2); (1, 4); (4, 1)\}$$

$$\omega_2 = \{(-1, -5); (-2, -1); (-1, -1); (-4, -4); (-4, -1); (-2, -6); (-2, -2); (-3, -1); (-3, -1); (-1, -3)\}$$

Figure 5 is an example where the minimum distance to class mean classifier fails. Since the

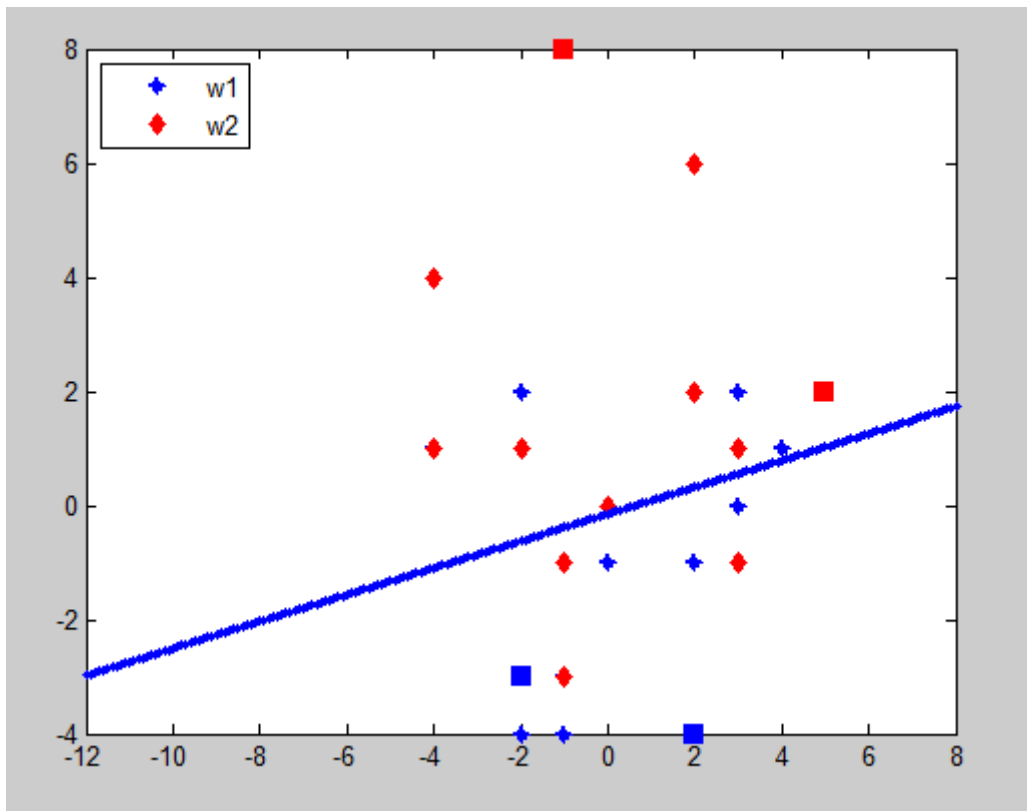


Figure 5: A bad example of minimum distance of class mean classifier

decision boundary is linear, we see that in this case the classifier does not do a very good job at classifying the points since the sample points of each class are scattered in the feature space. the dataset used for this is

$$\omega_1 = \{(2, -1); (3, 0); (3, 2); (-1, -3); (4, 1); (-2, -4); (0, -1); (-2, 2); (-1, -4); (-4, 1)\}$$

$$\omega_2 = \{(0, 0); (-2, 1); (-1, -1); (-4, 4); (-4, 1); (2, 6); (2, 2); (3, 1); (3, -1); (-1, -3)\}$$

CONCLUSION

The minimum distance to class mean classify is a simple classifier that uses very simple calculations. However the misclassification rate of this classifier is relatively higher since the decision boundary is linear.